Python for computational Problem Solving · Unit_03 · Unit 3: Functions, Functional Programming and Modules

DRAFT — UE26CS151A · Unit_03 · Unit 3: Functions, Functional Programming and Modules · Recursion

Standalone video lesson · authored study notes with source citations

Hugging Face PESU_Materials source tree, official units.json catalogue, transcript sidecars, slide text/PDF/PPT sources and the selected First_Year_8enterprise archive; see the provenance datasets in the release ZIP. Generated 18 October 2026.

· How to use this pack

Read the topic explanation, worked examples, and recap. Evidence citations point to source transcript intervals and slide pages; full slide images remain in the source visual index. Topics include the applicable parts: overview, key terms, formulas, figures, flowcharts, important points, worked examples, past-paper questions and a recap. IDs are shared with the separate documents: E = formula, G = graph, F = flowchart, W = worked example. Each ID is numbered by topic, so E4.2 is the second formula of topic 4.

Source text	Verbatim or OCR-derived text, kept separate from editorial notes.
Candidate	Automated topic/source association requiring manual confirmation.
Low ASR confidence	Transcript segment requires audio verification.
DRAFT	Human review remains incomplete.

Contents

1. 1 · Recursion
2. Appendix A · Symbol glossary
3. Appendix B · Sources, coverage and gaps
4. Appendix C · Revision and retrieval practice

M1

Recursion

1 Recursion

Study focus: Recursion. This topic explains the central idea, its conditions, and how the lecture applies it in context. Review the selected source statements and follow their citations for the full lesson segment. EV0000002 · EV0000003

SHORT-NOTE RECAP

Core explanation · 00:00–04:59

In today's class, we are going to be looking into the concept called as recursion.

Key details

- Recursion is basically a process where a function calls itself in order to solve a problem.
- It's called.
- This is called as a recursion.
- There are few recursive functions or sometimes recursive functions are so useful that they make it very easy for us to write a code.
- And there are few algorithms where it is very easy to express in terms of a recursive paradigm rather than writing it as an iterative process.
- But make sure that whatever is required and the best approach in order to solve your requirement, make use of it.
- In order to write a recursive program, I need a recursive case.
- I say recursive case.
- I have to mention what is my recursive case.
- At the same time, I also need to specify something called as a base case.
- Hopefully don't mind with my spelling mistakes, base case, or I can also call it as a termination case.
- Also called as a termination case.
- What will happen if I do not have a base case?
- If you do not mention where this particular function should stop executing, if you do not mention the function gets called again and again and you end up in an infinite loop.
- That's the reason why whenever you are writing a recursive function, make sure you write a recursive case at the same time, a very important one which is called as a base case.

- The main advantages of recursion is it is simple in nature.
- And it is simple in nature in the sense the code looks so simple.
- Code looks simple.
- At the same time it is the moment it is simple in nature, it also looks elegant.
- And the second advantage is basically your task can be broken down into a simpler subtlask.
- Task is broken down.
- [ASR low confidence; check audio].
- The set task.
- Is broken down into subtask.
- These are the advantages of your recursion, and the disadvantages are.
- Let me just say this as advantage so it is easy for us to go through.
- And what are the disadvantages?
- The disadvantages are.
- Lots of memory.
- It consumes lots of memory.
- It becomes inefficient.
- At the same time, it also consumes lots of time.
- And if you do a mistake, very tough to debug.
- Because you never know where the mistake is happening.
- And understanding the logic behind the recursion is not so easy.
- Let me quickly iterate over the advantages.
- Code looks simple and elegant.
- Task is broken down into the subtasks.
- Disadvantages is it takes lots of time, at the same time it also consumes lots of memory.
- And programs written using recursion are very hard to debug and understanding the logic behind recursive program is not easy.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000002 · EV0000003 · EV0000004 · +1 source units in pack.json

Core explanation · 05:00–09:59

Very famous problem we usually write.

Key details

- What is this what do I want to find out?
- I want to find out what is a factorial of a particular number.
- I will say if the number that I give you if it is equal to zero, which means you want to find a factorial of zero.
- Because factorial of zero is one.
- Otherwise, what I will say else, I will also say it as what?
- Instead of doing that, perform X into fact of X minus one.
- Every time you're trying to find out the factorial, what I'm trying to basically do, I am trying to find the value and the previous one, x minus one.
- I have to return this, so I will say it as what?

- factorial x into factorial of x minus one.
- My function is ready.
- What's the next thing that I have to do?
- I have to call this function.
- You will do the every time okay, maybe we will take a simple one, factorial of three.
- Three into factorial of two minus one I will do, then followed by two into factorial of the previous number.
- I've kept everything ready so that one second I will.
- Any time in this fine.
- Let me change the number.
- Factorial of five.
- And the expected answer is 120.
- And if you can see this is a name of a function, and this function factorial is again calling itself.
- This is fine.
- And I call this as a base case because when I say x becomes equal to 0, what it has to do?
- It has to return one.
- To whom it has to return?
- It has to return to a calling function.
- and we'll use a Python tutor.
- And then you have to say it as visualize the execution.
- Hope you have already learned this.
- Just go through this.
- And if you see the number of lines of code that I have written is 1, 2, 3, 4, 5, and 6.
- And how many number of steps that are involved here?
- 26 different steps are involved.
- Involved in order to execute the factorial program.
- That is what I said.
- It takes the amount of time it takes in order to execute is more.
- At the same time, internally, so many states have to be stored.
- Factorial function because if you can if you had seen your cursor, let me show this.
- The cursor is pointing at the beginning, the function header, okay?
- It basically processed the function header.

Example or application from the lecture

- So let me take up the same example.
- Let's quickly execute this program and find out what is the answer for this.
- Let's find out are there any errors or there are no errors, we'll find out.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000006 · EV0000007 · EV0000008 · +1 source units in pack.json

Core explanation · 10:00–14:59

I will get into the function body only when a particular function is called.

Key details

- And I'm now I'm trying to call what?
- Factorial of five.
- No, it is not equal to zero.
- If it is not equal to zero, what do I need to do?
- Go to your else part.
- What is x?
- X is 5, factorial of x minus 1 would be the previous number.
- 4 my sorry 5 minus 1, which is going to be 4.
- So go ahead with the next one and see.
- X becomes three.
- X becomes two.
- X becomes 1.
- The next time your X becomes 0.
- The moment your X becomes a 0, what will happen?
- It returns 1.
- It returns 1 to whom?
- To the one that has called.
- 2 into 3 was 6.
- 24 into 5 would become 120.
- I will tell you any number of programs that you do until and unless you code the problems.
- writing a recursive programs would be definitely tougher.
- Hope you have understood the basic concept and in the actual classes we will try to work with a few other programs.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000010 · EV0000011 · EV0000012 · +1 source units in pack.json

A Symbol glossary

Symbol	Name	Meaning	Appears in

B Sources, coverage and gaps

Source	Path	Use
Recursion transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/001_Recursion__57028129e466db02.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Recursion transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/001_Recursion__57028129e466db02.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Recursion transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/001_Recursion__57028129e466db02.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9997; only the JSON segmentation supplies the displayed timestamps.
Recursion transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/001_Recursion__57028129e466db02.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Historical question-source excerpts are shown only when a page has a strong text-based unit match. They are source references, not new solutions or exam predictions.

Known gaps, conflicts and corrections

1. 1 video-to-slide associations were not promoted to a match. Candidate rankings are listed in the transcript corpus and video-slide-matches.csv; the slide decks appear separately.
2. 4 transcript segments have low ASR confidence; they are flagged with timestamps in the transcript blocks and indexed in the corpus.
3. This is a visibly labelled source-preservation draft. Human page-level semantic and visual review is not complete; no unsupported subject summary or formula correction is claimed.
4. The 19.26 GiB MP4 files remain hosted on Hugging Face; the full video path/hash and transcript coverage are indexed. The full audio/video was not watched end-to-end.
5. Formula candidates are machine-extracted text with source locators; use the adjacent original slide page and the formula index before treating a candidate as an authoritative equation.

6. The study PDF contains edited, topic-organized lecture notes and textual slide-page takeaways. Full-page slide images remain in the source/visual index datasets, not embedded as the lesson. Human content, ASR, formula, and page-level visual review remains pending.

Coverage

Evidence is summarized by source to keep the reading copy legible. Full source-unit IDs, locators and decisions remain in pack.json.

Source	Unit counts	Example decisions
SRC00002	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0023. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0024 to s0041. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0042 to s0050. Sidecar variants compare...
SRC00003	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0023. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0024 to s0041. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0042 to s0050. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00004	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0023. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0024 to s0041. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0042 to s0050. Sidecar variants compare...
SRC00005	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0023. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0024 to s0041. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0042 to s0050. Sidecar variants compare...

C Revision and retrieval practice